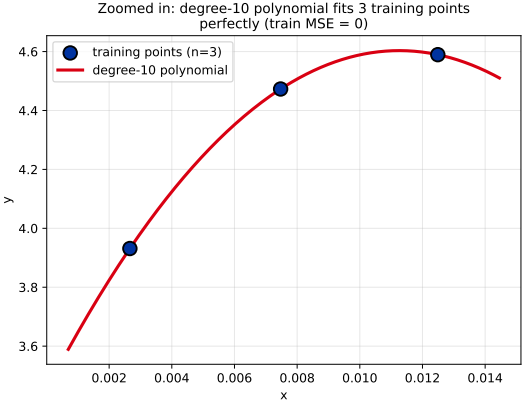
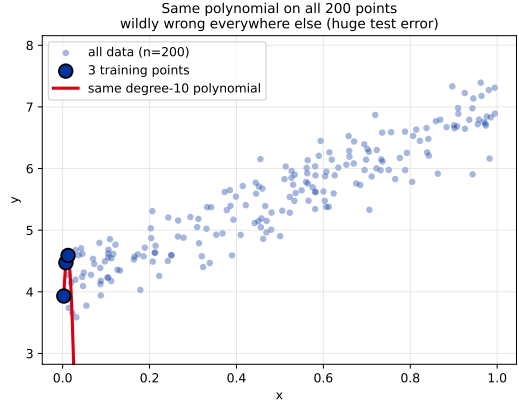
Train MSE = 0. Trust the model?





Today: how to tell when our model is fooling us, and how to evaluate it honestly.

Outline

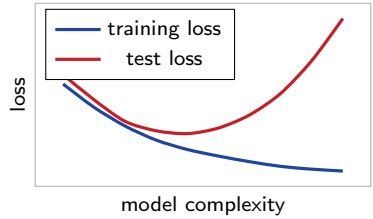
Why we need to hold data out

We fit a model by minimizing empirical risk on the training data (L01, L01b). The fitted model is the one with the lowest training loss.

Problem: the training loss is also the thing we used to choose the model. We're *grading the model on the questions we wrote the answer key from*.

We need an **independent estimate** of how the model will do on data it hasn't seen.

The picture every ML student should have in mind:



Training loss keeps falling. Test loss has a sweet spot, then climbs.

Two ways a model can be wrong

Before diving into how we detect this, let's name the two failure modes.

Underfit (high bias). The model is too simple to capture the pattern. Both training and test errors are large; the model is missing real structure.

Example: predicting house prices using only the city name – the data is fine, the model is too crude to use it.

Overfit (high variance). The model captures the training data too tightly, including its noise. Training error is tiny; test error is huge.

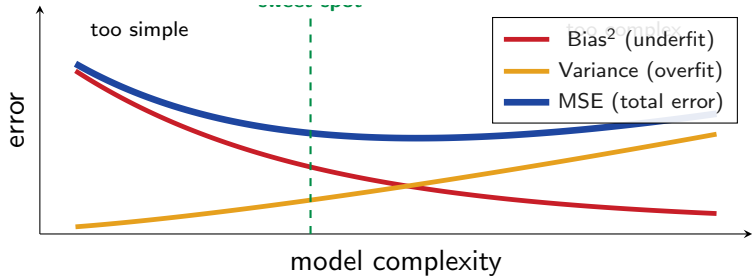
Example: a degree-10 polynomial fit to 3 data points – the curve weaves through each point exactly, but predicts garbage between them.

Both are bad. The art of ML is finding the sweet spot between them.

Bias-variance tradeoff

These two failure modes have formal names you already met in stat lectures (`03_stat`): **bias** and **variance**, with the identity

$$\mathrm{MSE}(\hat{\theta}) \;=\; \mathrm{Bias}^2(\hat{\theta}) \;+\; \mathrm{Var}(\hat{\theta})$$



Bias falls as the model gets richer; variance rises. MSE bottoms out somewhere in between. *The whole job of validation is to find that sweet spot empirically.*

Bias-variance is everywhere in ML

Same tradeoff, different knob:

Model class	**Too simple (high bias)**	**Too complex (high var)**
Polynomial regression	degree 1 (line)	degree 20 (wiggly, like Demo 1)
KNN	large k (oversmoothed)	$k = 1$ (memorizes noise)
Decision tree	shallow tree	deep tree, no pruning
Neural network	too few neurons	too many neurons, no dropout
Regularization	strong penalty (λ large)	no penalty ($\lambda = 0$)

Key insight. In every case the total error bottoms out at an intermediate complexity. This is exactly why we need **held-out test sets**, **cross-validation**, and (later) **regularization** – they're three different empirical ways to find that sweet spot.

Historical examples

Overfit: Babylonian astrology.
For centuries, astrologers built detailed rules connecting planetary positions to historical events: *"When Mars is in Taurus and the moon is half full, there will be a great harvest if the newborn prince is left-handed."*
They squeezed meaning out of every alignment – coincidences and noise treated as signal.

Underfit: medieval bloodletting.
For centuries, European doctors had one rule: *"Something's wrong? Drain blood."* Used for headaches, fevers, plague, melancholy alike – a single too-simple rule for every disease the model couldn't tell apart.

The art of ML is in the middle: complex enough to match real structure, simple enough not to chase noise.

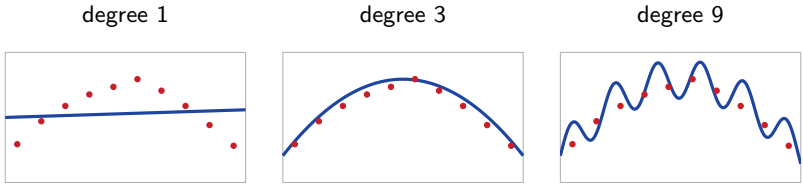
Kargin Haxordum examples

- Serjant Soghomonyan
- Ete qarakusi a meju klor a

TODO: explain which is overfit, which is underfit – to be filled in before delivery.

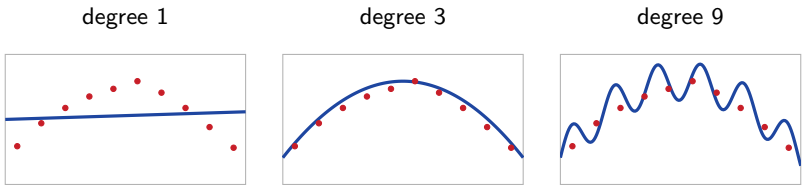
Predict-first: can a degree-9 polynomial fit 10 points perfectly?

You have 10 noisy samples from $y = \sin(\pi x) + \varepsilon$. You fit polynomials of degree 1, 3, and 9. Reminder from L01b:



Predict-first: can a degree-9 polynomial fit 10 points perfectly?

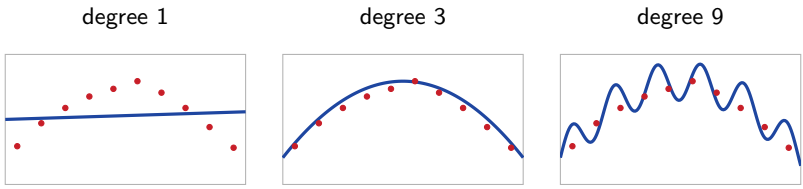
You have 10 noisy samples from $y = \sin(\pi x) + \varepsilon$. You fit polynomials of degree 1, 3, and 9. Reminder from L01b:



Predict: which has the smallest *training* error? Is that the model you should deploy?

Predict-first: can a degree-9 polynomial fit 10 points perfectly?

You have 10 noisy samples from $y = \sin(\pi x) + \varepsilon$. You fit polynomials of degree 1, 3, and 9. Reminder from L01b:



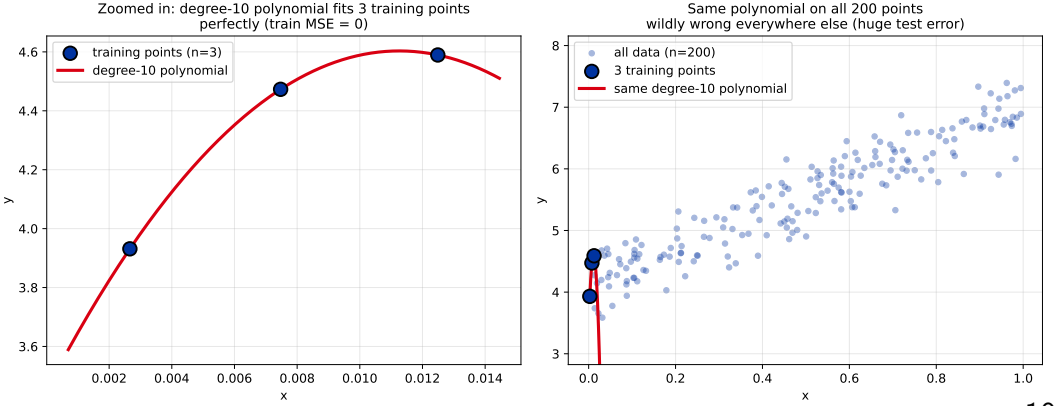
Predict: which has the smallest *training* error? Is that the model you should deploy?

Degree 9 has the smallest training error (it interpolates every point). It is also the *worst* model. Training error is not a measure of model quality.

Demo: degree-10 polynomial on 3 points

Callback: HW2's polynomial bonus showed train R^2 climbing while test R^2 dropped at $d = 3$ – the same fingerprint, here pushed to the extreme.

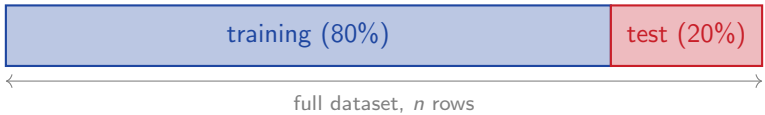
Take 3 random points from a noisy linear dataset. Fit a degree-10 polynomial. **Train MSE = 0.** Now predict on the full 200-point dataset:



The fix: hold out data the model never saw

Split the data into two pieces:

- **Training set** - used to fit the model.
- **Test set** - used *once* at the very end to estimate generalization.



Typical splits: 80/20, 70/30 for small data, 90/10 for large data.

Tradeoff: smaller test set $\Rightarrow$ more data to train on, but noisier estimate. Bigger test set $\Rightarrow$ honest estimate, but less training data. The right ratio depends on how much data you have.

Parameters vs. hyperparameters

Before we go further, one piece of vocabulary that the rest of this lecture (and the rest of the course) leans on:

Parameter θ. A value the model *learns* from data during training.

- Linear regression: the coefficients $\theta_0, \theta_1, \ldots$
- Neural net: every weight.

Found by minimizing the empirical risk (L01, L01b).

Hyperparameter. A setting *you* choose before fitting. The model can't learn it because changing it would change the model class itself.

- Polynomial regression: the degree d.
- Gradient descent: the learning rate α.
- KNN: the number of neighbors k.
- Ridge / Lasso (next chapter): the penalty λ.

Why this matters here: validation exists to help us pick hyperparameters honestly. "Try a few values, look at the score, pick the best" is the whole game for the next several frames.

Mechanics: sklearn train_test_split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # fraction held out
    random_state=509,       # default seed (see CLAUDE.md)
    shuffle=True,           # default True; disable for time series
)
# For classification, add stratify=y to preserve class ratios (see
    Section 5).
# >>> X_train.shape, X_test.shape
# ((160, 5), (40, 5))
```

- `test_size=0.2` $\Rightarrow$ 80/20 split.
- `random_state` fixes the seed - same split every run. Without it, debugging becomes impossible.
- `stratify=y` for classification: preserves the class ratio. Formalized later as Stratified CV.
- `shuffle=False` for time-series: keep chronological order (you can't train on tomorrow to predict today).

Critical rules

Ranked by how often students miss them:

1. **Don't iterate on the test set.** The most-violated rule. If you make any modeling decision based on test-set behavior, the test set becomes part of training. Look at test *once*, at the end.
2. **Split before any preprocessing.** The leakage rule from L01c (*fit on train, transform on train and test*) requires you to have split first. Without this, mean-imputation and scaling silently leak.
3. **Group-structured data needs group splits.** If you have 10 photos per person and randomly split, the same person appears in both train and test. The model memorizes faces, not features. Use `GroupShuffleSplit`.
4. **Time series needs chronological splits.** A random shuffle leaks the future into the past. Use `TimeSeriesSplit` or a single chronological cutoff.

What about EDA? Distribution-level facts (sizes, classes present, ranges) are fine to check on the full data before splitting. The rule is about *modeling decisions* and *performance numbers*, not about awareness.

Nuance: when the rows aren't independent

Random shuffle assumes every row is exchangeable. Two cases where this is wildly wrong:

1. Multiple records per entity (patients, users, photos).

100 patients $\times$ 5 visits $=$ 500 rows. Random 80/20 puts $\sim$ 4 visits of the same patient in train and $\sim$ 1 in test $\Rightarrow$ model memorizes *the patient*, not the disease pattern.

```
from sklearn.model_selection import GroupKFold
GroupKFold(n_splits=5).split(X, y, groups=patient_id)
```

2. Time-ordered data (forecasting, sensor readings, prices).

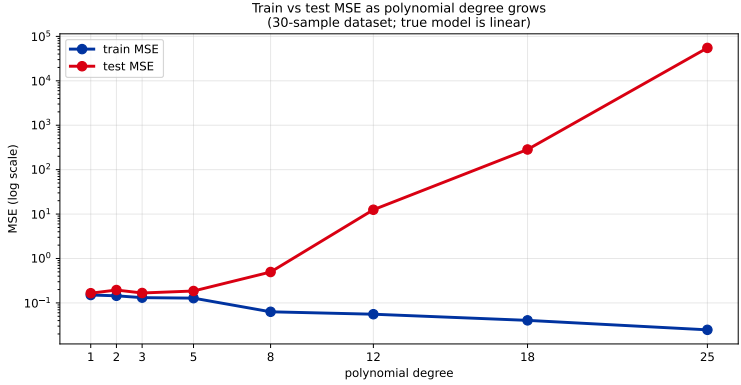
Random shuffle puts tomorrow's row in training and yesterday's row in the test set – training on the future to predict the past.

```
from sklearn.model_selection import TimeSeriesSplit
TimeSeriesSplit(n_splits=5)  # expanding-window CV; or a chronological
    cutoff
```

Diagnosis: (a) no group ID in both train and test, (b) no test timestamp before the latest train timestamp. Violate either $\Rightarrow$ leaking.

Demo: train vs test MSE as polynomial degree grows

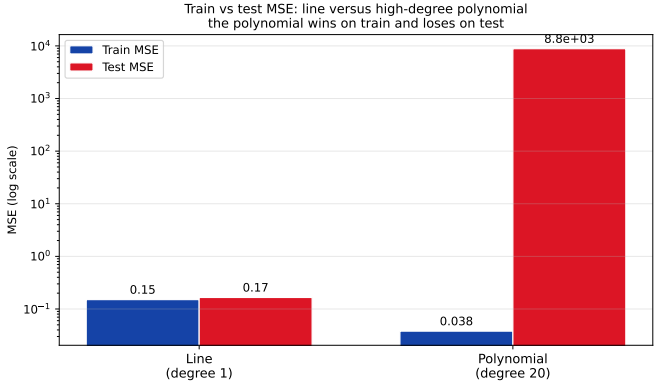
30 noisy samples from a linear truth ($y = 4 + 3x + \text{noise}$). Fit polynomials of degree 1 to 25 on the training split. For each, measure train MSE and test MSE on the test split.



Train MSE drops monotonically. Test MSE bottoms out around degree 1–3 (matching the true model), then explodes. *The train-test gap is the empirical fingerprint of overfitting.*

Demo: line versus high-degree polynomial

Same 30-sample dataset. Two models: a straight line, and a degree-20 polynomial. Train MSE and test MSE side by side:



The polynomial wins on training (much lower MSE) and loses catastrophically on test. *If you only looked at the train bars you would pick the polynomial. That is exactly the trap that train/test split exists to prevent.*

The hyperparameter tuning trap

After splitting, you try several models or hyperparameter settings. You evaluate each on the test set, then pick the best.

Common workflow. Let's check whether the reported number is honest.

Each comparison *uses* the test set. If you pick the configuration with the best test score, you've absorbed the test set into your model-selection process.

Analogy. You take a practice exam 100 times until you score perfectly. Do you expect to score perfectly on the real exam?

Predict-first: tuning on the test set

You try 50 different polynomial degrees on the same train/test split. You pick the degree with the smallest test MSE and report that MSE as your model's quality.

Predict-first: tuning on the test set

You try 50 different polynomial degrees on the same train/test split. You pick the degree with the smallest test MSE and report that MSE as your model's quality.

Predict:

- Is the reported MSE an unbiased estimate of how the model will perform on *new* unseen data?
- By roughly how much does the reported MSE under-estimate the true generalization error?

Predict-first: tuning on the test set

You try 50 different polynomial degrees on the same train/test split. You pick the degree with the smallest test MSE and report that MSE as your model's quality.

Predict:

- Is the reported MSE an unbiased estimate of how the model will perform on *new* unseen data?
- By roughly how much does the reported MSE under-estimate the true generalization error?

Answers:

- **No.** The chosen configuration happened to do well on *this specific test set*. Some of that performance is noise that won't reproduce on truly new data.
- **Typically 10-40% optimistic** on small-to-medium datasets when tuning over ~ 50 configurations. The bias grows with the number of comparisons and shrinks with test-set size. On a fresh test set the reported model would score worse than reported.

Fix: use a separate **validation set** for tuning. Touch the test set only once.

The three-way split

Split the data into *three* pieces.

Workflow:

1. Fit each candidate model on training.
2. Evaluate each on validation. Pick the best.
3. Optionally *refit* the chosen configuration on training + validation (more data = better fit).
4. Evaluate **once** on test. Report that number.

Step 3 is common but not universal - some practitioners skip the refit and report the model that was actually tuned on validation. Both views are defensible.

The discipline: test set looked at exactly once

The hardest part isn't the code. It's the discipline:

- Don't make modeling decisions based on test scores.
- Don't engineer features after seeing test set behavior.
- Don't iterate on hyperparameters after seeing the test score.
- Don't compare models on test and pick the best.

Treat the test set like a sealed envelope you open at the end. Awareness of the test set (sizes, class balance) is fine - making decisions based on it is not.

Kaggle terminology: the test set is the "private leaderboard". Touch it more than once during development and your real-world performance will lag your reported performance.

One validation set is noisy

You have 200 Yerevan listings. 60/20/20 split $\Rightarrow$ 40 validation rows.

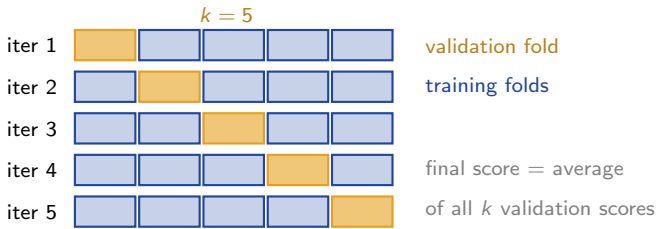
Model A scores 0.81 MSE on validation, model B scores 0.79. Is B really better, or is the gap just noise on 40 samples?

Same data, different random split can flip which model looks better. On a 200-row dataset the validation score has a standard error of several percent.

Solution: use k different validation sets and average their scores. Averaging k roughly-independent estimates reduces the variance of the mean by a factor of $\approx 1/k$ (CLT-style, math module 20). This is k-**fold cross-validation**.

k-fold cross-validation: how it works

Split the (non-test) data into k equal chunks called *folds*. For each of k iterations, hold out one fold as validation and train on the rest.



Subtle point: you train k different models (one per iteration), not one. The CV score estimates how well the *training protocol* generalizes, not any specific model's performance. The final model you ship is usually fit on *all* the training data after CV picks the hyperparameters.

Choosing k and the cost

Standard choices:

- $k = 5$ - the default. Gives 80/20 train/val per fold (matches standard hold-out), bias is low, compute manageable.
- $k = 10$ - lower variance, 2x more compute. Use when you can afford it.
- $k = n$ - leave-one-out (LOO). Train n models. Used only when n is tiny; the LOO score has surprisingly high variance because folds are nearly identical.
- `RepeatedKFold(n_splits=5, n_repeats=3)` - repeats k-fold with different random splits and averages. Lower variance when compute allows.

When CV is worth it:

- Small datasets ($n < 10{,}000$) - one validation set is noisy.
- Model selection / hyperparameter tuning.
- Reporting a confidence interval on the score (mean $\pm$ std over folds).

When a single split is enough:

- Big data ($n > 10^6$) - one split is already low-noise.
- Quick prototyping iterations.
- Compute is the bottleneck.

LOOCV: the boundary case $k = n$

Push k all the way to n: every fold leaves out exactly one row. With n training rows you fit n models.

Why it sounds appealing

- Uses $(n-1)/n$ of the data each fold $\Rightarrow$ almost-unbiased.
- Deterministic: no random fold assignment.

Why it rarely beats $k = 5$ or 10

- Compute: train n models. Brutal for any non-trivial model.
- Folds are nearly identical training sets $\Rightarrow$ scores are highly correlated $\Rightarrow$ a misleading variance estimate.
- Empirically, $k = 5$ or 10 gives a better bias-variance tradeoff for the CV estimate itself.

```
from sklearn.model_selection import LeaveOneOut, cross_val_score
cross_val_score(model, X, y, cv=LeaveOneOut())
```

Use when: n is tiny (say < 50) and a single held-out fold is too noisy. Otherwise: $k = 5$ or 10.

CV in code

sklearn footgun: `cross_val_score` *maximizes* by convention. Error metrics are negated (`neg_mean_squared_error`). Always flip the sign when reporting.

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import Ridge

kf = KFold(n_splits=5, shuffle=True, random_state=509)
scores = cross_val_score(
    Ridge(alpha=1.0), X_train, y_train,
    cv=kf,
    scoring="neg_mean_squared_error",
)
print(f"MSE = {-scores.mean():.3f} +/- {scores.std():.3f}")
```

- **Mean and std across folds** give a sanity check on stability. *Always report both.*
- Pass `X_train`, not the full dataset - test set stays untouched.
- For out-of-fold predictions (used in stacking, calibration): `cross_val_predict` returns one prediction per row.
- Multiple metrics at once: `cross_validate(..., scoring=["neg_mean_squared_error", "r2"])` returns a dict.
- Lower-variance estimates: `RepeatedKFold(n_repeats=3)` averages over multiple shuffles

Pipeline + CV: how the leakage rule survives folds

CV trains k models. If you fit your preprocessor (scaler, imputer, encoder) on the full dataset *before* the CV loop, you leak into every fold. The clean way: wrap preprocessing + model in a `Pipeline` and pass the pipeline to `cross_val_score`. sklearn refits the preprocessor on each fold's training rows only.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score

pipe = Pipeline([
    ("scaler", StandardScaler()),      # refit per fold
    ("lr",     LinearRegression()),
])
cv = KFold(n_splits=5, shuffle=True, random_state=509)
scores = cross_val_score(pipe, X_train, y_train, cv=cv,
                         scoring="neg_mean_squared_error")
```

HW2 used `ColumnTransformer + LinearRegression`; that pipeline drops straight into `cross_val_score` – encoders, scaler, imputer all refit per fold, leakage rule enforced for free.

Demo: what happens when you leak

Toy demo: 100 rows, 1 numeric feature, scale it. Two ways:

```
# WRONG: fit scaler on ALL data, then split
scaler = StandardScaler().fit(X)            # <-- leaks test stats
Xs = scaler.transform(X)
cross_val_score(LinearRegression(), Xs, y, cv=5)
# -> CV MSE = 0.142   (optimistic, the scaler "saw" the test rows)

# RIGHT: scaler lives inside the Pipeline, refit per fold
pipe = Pipeline([("sc", StandardScaler()), ("lr", LinearRegression())])
cross_val_score(pipe, X, y, cv=5)
# -> CV MSE = 0.158   (honest, +11% higher)
```

On this toy dataset the gap is small ($\sim 11\%$). For `target encoding` or `KNNImputer` (which use the target or whole-row similarity), the gap can be 5–10 percentage points of R^2. Same rule, far worse consequence.

Diagnosis: if your CV score looks suspiciously good, check whether *anything* fit before the CV loop touched all the rows. Imputers, scalers, encoders, target encoders, PCA – all common culprits.

Always have a dumb baseline

A CV-mean MSE of 0.16 sounds good. Is it?

```
from sklearn.dummy import DummyRegressor
baseline = DummyRegressor(strategy="mean")      # predicts y.mean() every
    time
cross_val_score(baseline, X, y, cv=5, scoring="neg_mean_squared_error")
# -> MSE = 0.91   (a real model has to beat this)

# Or for classification:
from sklearn.dummy import DummyClassifier
DummyClassifier(strategy="most_frequent")       # predicts the modal class
```

Sounds obvious – almost no one does it. "Mean predictor" $R^2 = 0$ *by construction*, and your model's R^2 is the part you can claim credit for.

Workflow rule: fit a dummy baseline first, every project. Then your fancy model has to clear that bar before you tune it. Saves you from celebrating models that aren't doing anything.

CV is noisy too – the 1-SE rule

Each fold gives one score. With $k = 5$ you get 5 numbers. Their mean is the headline; their spread tells you how much to trust it.

Example: polynomial regression on the 30-point dataset from earlier, 5-fold CV across polynomial degree:

degree d	CV mean MSE	CV std
1	0.18	0.05
2	0.17	0.06
3	**0.16**	0.07
4	0.19	0.10
6	0.31	0.18
9	1.42	0.85

Headline winner: $d = 3$ (mean 0.16). But its CV std is 0.07 – the gap to $d = 1$ (0.18) is smaller than the standard error.

The 1-SE rule (Tibshirani / Hastie): pick the *simplest* model whose CV mean is within 1 standard error of the best. Here that's $d = 1$ – the line. Same fit quality within noise, half the parameters. Prevents complexity creep when CV is noisy.

The imbalanced-data problem

Many real classification tasks are imbalanced:

- Credit-card fraud: 0.2% fraudulent transactions
- Disease screening: 1% positives in screened population
- Click-through prediction: 2% click rate
- Apartment rent fraud detection: maybe 5% suspicious listings

Random k-fold CV draws folds at random. With a 5% positive class, the positives are scattered uniformly - but *uniformly* doesn't mean *evenly*. Some folds get more, some fewer, sometimes zero.

Why care? A fold with 0% positives breaks the protocol in two ways: the model trained without it never sees the rare class, and the score from that fold isn't meaningful.

Next frame: how often does this actually happen?

Predict-first: 5-fold CV on 5% positives

You have 200 rows of which 10 are positive (5%). You run vanilla 5-fold CV.

Predict-first: 5-fold CV on 5% positives

You have 200 rows of which 10 are positive (5%). You run vanilla 5-fold CV.

Predict:

- How many positives does each fold get *on average*?
- What's the probability that at least one fold has zero positives?

Predict-first: 5-fold CV on 5% positives

You have 200 rows of which 10 are positive (5%). You run vanilla 5-fold CV.

Predict:

- How many positives does each fold get *on average*?
- What's the probability that at least one fold has zero positives?

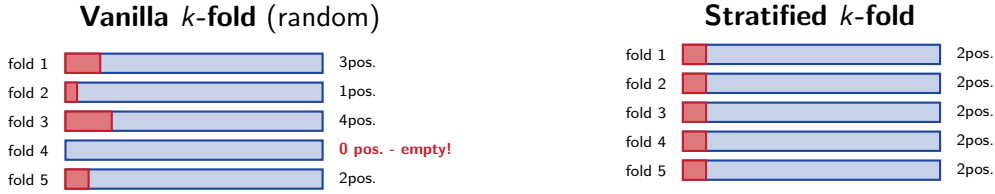
Answers:

- **On average 2 positives per fold** (10 split across 5).
- **Roughly 6% chance** at least one fold is empty - the 10 positives can cluster by chance. (Each fold's probability of being empty is $\binom{190}{40}/\binom{200}{40} \approx 0.8^{10} \approx 11\%$; with 5 folds, $\approx 1-(1-0.11)^5 \approx 44\%$ if independent, but folds are negatively correlated $\Rightarrow$ true probability is $\approx 6\%$.) With smaller imbalance (1% positives, $n = 200$) it's nearly guaranteed.

Solution: split each class separately, so every fold ends up with its share of the minority class. **Stratified k-fold.**

Stratified k-fold: vanilla vs stratified

Same data, two CV strategies. Each row is one fold's positive count (out of 10).



Stratified k-fold splits each class separately. Every fold receives its share. No more empty-positive folds.

For classification: default to `StratifiedKFold`. There's almost no reason to use plain `KFold` on classification data. The sklearn helpers (`cross_val_score`, `GridSearchCV`) auto-stratify if `y` is categorical, but be explicit.

Stratified CV in code

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=509)
scores = cross_val_score(
    LogisticRegression(class_weight="balanced"),  # separate trick
    X_train, y_train,
    cv=skf,
    scoring="f1",        # not accuracy
)
print(f"F1 = {scores.mean():.3f} +/- {scores.std():.3f}")
```

- Use `StratifiedKFold` *by default* for classification. Almost no reason to use plain `KFold`.
- Pair with the right metric. F1 / ROC-AUC / PR-AUC handle imbalance honestly; accuracy doesn't.
- `class_weight="balanced"` is a *model* parameter (orthogonal to stratification) that tells the fit step to weight rare classes more. Commonly paired with stratified splitting on imbalanced data.

Nested CV – honest evaluation when you also tune

When you pick a hyperparameter (say polynomial degree d) with CV *and* evaluate with the same CV, you've used the validation procedure to make a modeling decision. The reported CV score is optimistically biased.

Fix: two CV loops.

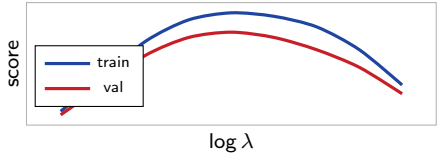
- **Outer loop** (K folds) – for evaluation. Each outer fold has its own held-out test slice.
- **Inner loop** (J folds, on outer-train) – for picking the hyperparameter (degree d, learning rate, etc.).
- Refit best HP on the full outer-train, score on outer-test. The outer scores are the honest estimate.

```
from sklearn.model_selection import KFold, cross_val_score
outer = KFold(n_splits=5, shuffle=True, random_state=509)
honest_scores = []
for tr_idx, te_idx in outer.split(X):
    X_tr, X_te = X[tr_idx], X[te_idx]
    # inner CV on the outer-train to pick the best degree
    best_d = pick_best_degree_via_cv(X_tr, y[tr_idx], degrees=[1,2,3,5,9])
    # refit on the whole outer-train, score on outer-test
    honest_scores.append(score_poly_fit(X_tr, y[tr_idx], X_te, y[te_idx],
        best_d))
print(f"honest CV MSE = {np.mean(honest_scores):.3f}")
```

Validation curves and learning curves

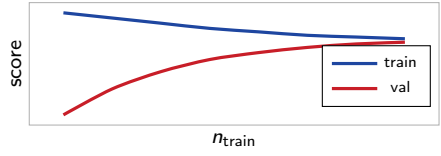
Two CV-based diagnostics every ML practitioner uses. Both plot CV score vs. a single axis.

Validation curve
(score vs. hyperparameter)



Diagnoses: under/overfitting per hyperparameter setting.

Learning curve
(score vs. train set size)



Diagnoses: would more data help? (Gap closing = yes.)

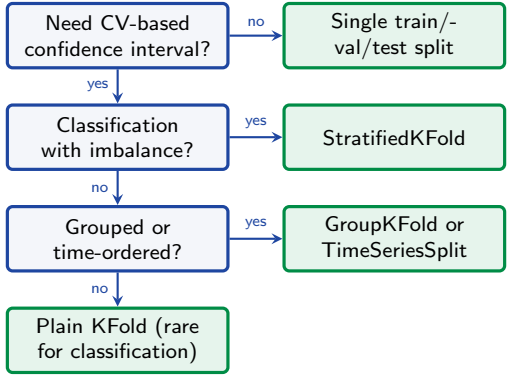
Both available as `sklearn.model_selection.validation_curve` and `learning_curve`.

What to do with each shape:

- Validation-curve sweet spot at the *edge* of your range ⇒ widen the hyperparameter range.
- Learning-curve gap *not closing* ⇒ bias-dominated; more data won't help. Change the model class (L01d2 callback).
- Learning-curve gap *closing slowly* ⇒ variance-dominated; more data WILL help.

Decision flowchart

Single train/val/test split is fine if you don't need a CI on your score - for example, big data, fast prototyping, or compute-constrained settings. CV is for when score variance matters.



Recap

- **Training error lies.** Always evaluate on data the model didn't see.
- **Train/test split.** Simplest fix. 80/20 typical. Split before any preprocessing (L01c leakage rule).
- **Validation set.** If you tune anything, you need a third set. Test set touched once at the end.
- **Cross-validation.** Rotates validation through k folds, reducing noise by $\approx 1/k$. Default $k = 5$. Use `cross_validate` for multiple metrics; `RepeatedKFold` for lower-variance estimates.
- **Stratified CV.** Default for classification - preserves class ratio per fold.
- **Group / Time splits.** Use when rows aren't independent. Otherwise the model memorizes entities or peeks at the future.
- **Validation/learning curves.** CV-based diagnostics for under/overfitting and "would more data help?".
- **Bias-variance lens (L01d2):** CV (and the choice of k) is itself a bias-variance tradeoff - small k is biased, large k is noisy. The framework is fractal.

Coming next: L05b - nested CV, the honest way to report a tuned model's generalization error.